

(A)

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

- 1."Low Salary": All monthly salaries strictly less than \$20,000.
- 2."Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].
- 3."High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

| Output Table   |                |
|----------------|----------------|
| category       | accounts_count |
| Low Salary     | 1              |
| Average Salary | 1              |
| High Salary    | 3              |

## Solution Query :

```
SELECT
    'LOW CATEGORY' AS CATEGORY,
    COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
FROM
    ACCOUNTS
WHERE
    INCOME <= 20000
UNION ALL
SELECT
    'AVERAGE CATEGORY' AS CATEGORY,
    COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
FROM
    ACCOUNTS
WHERE
    INCOME > 20000 AND INCOME <= 50000
UNION ALL
```

```

SELECT

    'HIGH CATEGORY' AS CATEGORY,

    COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT

FROM

    ACCOUNTS

WHERE

    INCOME > 50000;

```

## (B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

Output Table:

| name    | dept_name | salary |
|---------|-----------|--------|
| Alice   | HR        | 50000  |
| Bob     | HR        | 25000  |
| Charlie | IT        | 80000  |
| David   | IT        | 40000  |
| Emma    | Sales     | 45000  |

## Part B: Apply 10% HR increase using IN subquery filtering

Updated Salaries State Output Table:

| emp_id | salary |
|--------|--------|
| 1      | 55000  |
| 2      | 27500  |
| 3      | 80000  |
| 4      | 40000  |
| 5      | 45000  |

**Solution Query :**

**(i)**

```
SELECT E.*, D.*, S.*  
  
FROM  
  
EMPLOYEE AS E  
  
JOIN  
  
DEPARTMENT AS D  
  
ON  
  
E.DEPT_ID = D.DEPT_ID  
  
JOIN  
  
SALARIES AS S  
  
ON  
  
E.EMP_ID = S.EMP_ID;
```

**(ii)**

```
UPDATE SALARIES  
  
SET SALARY = SALARY + SALARY * 0.10  
  
WHERE EMP_ID  
  
IN  
  
(  
  
SELECT E.EMP_ID  
  
FROM EMPLOYEE AS E  
  
JOIN  
  
DEPARTMENT AS D  
  
ON  
  
E.DEPT_ID = D.DEPT_ID  
  
WHERE D.DEPT_NAME = 'HR'  
  
)
```

(C)

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza\_toppings. Every customer must choose **exactly 3 different toppings** to prepare a pizza. Write a PostgreSQL query to generate **every possible 3-topping pizza combination** along with its **total ingredient cost**.

**Requirements**

Every pizza must contain **exactly 3 different toppings**.  
The same topping **cannot appear more than once** in a pizza.  
Different arrangements of the same toppings should be considered the **same pizza**.

Example:

- Chicken, Pepperoni, Sausage
- Pepperoni, Chicken, Sausage
- Sausage, Chicken, Pepperoni

These are all the same pizza, so only **one** should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each pizza.

Sort the output:

- First by **highest total cost**
- If two pizzas have the same cost, sort them alphabetically.

| Output Table                   |            |
|--------------------------------|------------|
| pizza                          | total_cost |
| Chicken,Pepperoni,Sausage      | 1.75       |
| Extra Cheese,Pepperoni,Sausage | 1.60       |
| Chicken,Extra Cheese,Sausage   | 1.65       |
| Extra Cheese,Onions,Sausage    | 1.35       |
| Chicken,Onions,Sausage         | 1.50       |
| Chicken,Extra Cheese,Pepperoni | 1.45       |
| Extra Cheese,Onions,Pepperoni  | 1.15       |
| Chicken,Onions,Pepperoni       | 1.30       |
| Chicken,Extra Cheese,Onions    | 1.20       |
| Extra Cheese,Onions,Sausage    | 1.35       |

**Solution Query:**

```
SELECT
    CONCAT(T1.TOPPING_NAME, ','T2.TOPPING_NAME, ',' T3.TOPPING_NAME) AS PIZZA,
    (T1.INGREDIENT_COST+T2.INGREDIENT_COST+T3.INGREDIENT_COST)AS TOTAL_COST
FROM PIZZA_TOPPINGS AS T1
JOIN
```

PIZZA\_TOPPINGS AS T2

ON

T1.TOPPING\_NAME < T2.TOPPING\_NAME

JOIN

PIZZA\_TOPPINGS AS T3

ON

T2.TOPPING\_NAME < T3.TOPPING\_NAME;